

From GET to Agentic Government

Timeline

1989–1993 — The Web and the earliest HTTP

Tim Berners-Lee submitted his first Web proposal at CERN in March 1989. By the end of 1990, the first browser and web server were operating on a NeXT computer. The software was released more widely during 1991, and on 30 April 1993 CERN made the Web technology available on a royalty-free basis. ([CERN](#))

The protocol later called **HTTP/0.9** was extremely small: essentially a request for a document followed by the document itself. It had no status codes, content types, authentication framework or rich metadata. HTTP was initially a document-retrieval protocol, not an API platform. ([W3C — HTTP/0.9](#))

Historical significance: openness and implementability mattered more than sophistication. The Web expanded because different organisations could implement the same simple protocol without obtaining permission.

1996 — HTTP/1.0 is formally documented

RFC 1945, published in May 1996, documented HTTP/1.0. It added MIME-like messages, headers, metadata, content types and richer request and response semantics. ([RFC Editor](#))

This was the point at which an HTTP exchange became more than “fetch this file”. A response could explain:

- what happened;
- what representation was returned;
- how the representation should be interpreted;
- whether the client should try something else.

That basic structure—method, resource identifier, headers, status and content—remains the foundation of most government APIs.

1997–1999 — HTTP/1.1 makes the Web scalable

The first standards-track HTTP/1.1 specification appeared as RFC 2068 in January 1997 and was revised as RFC 2616 in June 1999. It strengthened caching, persistent connections, intermediaries, content negotiation, virtual hosting and extensibility. It also described HTTP as a generic, stateless application-level protocol suitable for distributed systems, rather than merely for transferring web pages. ([RFC Editor](#))

Government relevance: HTTP’s methods, status codes, caching semantics and intermediaries gave public services a shared architectural language. Gateways, proxies, content delivery networks, audit components and security controls could participate without every system sharing an implementation.

1998–2001 — Two traditions emerge: web services and REST

XML 1.0 became a W3C Recommendation in February 1998. SOAP 1.1 followed in May 2000 and WSDL 1.1 in 2001. Together, XML, SOAP and WSDL enabled highly structured messages, operation-oriented service definitions, generated clients and extensive enterprise integration tooling. ([w3.org](#)) SOAP 1.1 ([W3C Note, 8 May 2000](#)); WSDL 1.1 ([W3C Note, 15 March 2001](#)).

In 2000, Roy Fielding’s doctoral dissertation formally described **Representational State Transfer**, or REST. REST was not a synonym for “JSON endpoints”. It was an architectural style involving constraints such as client–server separation, statelessness, cacheability, layering and a uniform interface. ([UCI Bren School](#))

A useful way to present this is:

- **SOAP/WSDL** concentrated on messages, operations and explicit service contracts.
 - **REST** concentrated on the architectural properties of a large, evolvable, distributed system.
- They addressed overlapping problems at different levels. “SOAP bad, REST good” is an inadequate historical account.

2000–2006 — APIs become platforms

eBay launched its first APIs in 2000 to allow sellers to manage their businesses at scale. They were initially SOAP-based and restricted to selected partners. This illustrates the early transition from APIs as internal integration mechanisms to APIs as the boundary of a commercial platform. ([eBay Inc.](#))

JSON subsequently provided a smaller, programming-language-friendly alternative to many XML payloads. The application/json media type was first defined by the IETF in RFC 4627 in July 2006, later superseded by RFC 8259 (the current JSON standard, STD 90). ([RFC Editor](#))

The dominant API model gradually became:

HTTP semantics + resource-oriented URLs + JSON representations + developer documentation.

This model was simpler than many enterprise web-service stacks, although simplification sometimes removed useful rigour around schemas, contracts and lifecycle management.

2012 — Delegated authority and API-first government

OAuth 2.0 was standardised in October 2012 (RFC 6749). Its central contribution was to allow a third-party application to receive limited access to an HTTP service, either on behalf of a user or in its own right, without simply sharing the user's password. ([RFC Editor](#))

The same year, [legislation.gov.uk](#) provided an important UK government example of API-first architecture. The legislation API was built first and the public website was then built on top of it. The API was described as the “beating heart” of the service rather than a later integration feature. ([Government Digital Service](#))

That distinction remains important:

API-first does not mean building an API before understanding the service. It means treating the service boundary as a first-class product and contract.

2014–2017 — The modern API ecosystem forms

In 2014, the large HTTP/1.1 specification was divided into a set of more focused RFCs. HTTP/2 was standardised in 2015 (RFC 7540), introducing binary framing and multiplexing while retaining HTTP's application semantics. ([RFC Editor](#))

In November 2015, the OpenAPI Initiative was formed under the Linux Foundation and the Swagger 2.0 specification was donated to it. OpenAPI 3.0 followed in 2017, establishing a vendor-neutral, machine-readable description format for HTTP APIs. ([openapis.org/about](#))

The period also produced several specialised alternatives:

- GraphQL had been used at Facebook since 2012 and was open-sourced in 2015, allowing consumers to request data through a typed query language. ([graphql.org](#))
- gRPC emerged as a high-performance RPC approach using Protocol Buffers, particularly suited to internal service-to-service communication and streaming. ([gRPC](#)) Source: [grpc.io](#).
- AsyncAPI was created in November 2017 to describe message-driven and event-oriented APIs. ([asyncapi.com](#))

The important change was **architectural pluralism**. “API” no longer implied a single interaction style.

2018–2019 — UK government standardisation and event interoperability

GDS first published its cross-government API technical and data standards in February 2018. The guidance promotes REST for many cases while recognising that GraphQL or gRPC may be better suited to particular needs. It also recommends specification-first development using OpenAPI. ([GOV.UK](#))

In 2019, the Government Open Standards Board formally recommended OpenAPI 3 for describing RESTful APIs. ([technology.blog.gov.uk](#))

CloudEvents reached version 1.0 in October 2019, providing common metadata for describing events across different platforms and transports. This addressed a recurring problem in event-driven systems: every producer defining its own idea of what an event envelope should contain. ([CNCF](#))

2021–2022 — HTTP semantics are separated from the wire format

The 2022 HTTP specifications provide a particularly useful conceptual lesson. RFC 9110 defines semantics shared by all HTTP versions; HTTP/1.1, HTTP/2 and HTTP/3 have separate specifications for representing and transporting those semantics. ([RFC Editor](#))

HTTP/3 maps the same HTTP semantics onto QUIC, supporting independent streams and avoiding some transport-level blocking associated with HTTP/2 over TCP. ([RFC Editor](#))

This means that:

GET, POST, status codes, headers, caching and resource semantics are the enduring contract. HTTP/1.1, HTTP/2 and HTTP/3 are different ways of carrying it.

That distinction is valuable when discussing government standards: **standardise durable semantics and outcomes, not incidental infrastructure choices.**

2023–June 2026 — Contracts expand from endpoints to events and workflows

The current specification landscape extends beyond individual API operations:

- AsyncAPI 3.0 was released in 2023, with AsyncAPI 3.1 following in January 2026. ([asyncapi.com](#))
- OpenAPI 3.2 was released in September 2025, adding capabilities including improved streaming descriptions and broader HTTP method support. ([openapis.org](#))
- Arazzo Specification v1.1.0 was published on 17 May 2026 and officially announced by the OpenAPI Initiative on 19 May 2026. It describes sequences of API calls and their dependencies in both human-readable and machine-readable form. ([OpenAPI Initiative Publications](#)) Source: [spec.openapis.org/arazzo \(v1.1.0\)](#).
- The OAuth 2.0 Security Best Current Practice was published in 2025 (RFC 9700), incorporating deployment experience and deprecating modes now considered insecure. ([RFC Editor](#))
- Model Context Protocol, or MCP, provides an emerging standard for connecting LLM applications to tools and data sources. ([Model Context Protocol](#))
- Agent2Agent, or A2A, was placed under Linux Foundation governance in June 2025 as an emerging protocol for communication and collaboration between AI agents. ([Linux Foundation](#))

MCP and A2A should currently be considered as **emerging protocols**, not settled government standards. They do not replace well-designed APIs. They make the quality, authority model and machine-readability of those APIs more consequential.

The historical movement in six steps

The timeline can be condensed into six architectural shifts:

1. **Documents to resources and capabilities**
HTTP moved from retrieving pages to supporting general distributed systems.
2. **Private integration to open ecosystems**
APIs became boundaries through which partners, suppliers and other public bodies could build independently.
3. **Prose documentation to executable contracts**
OpenAPI and related specifications enabled validation, generated documentation, testing and client generation.
4. **Synchronous requests to events and streams**
AsyncAPI and CloudEvents recognised that many government interactions are notifications and state changes, not immediate question-and-answer exchanges.
5. **Individual operations to whole outcomes**
Workflow specifications such as Arazzo describe how several operations combine to achieve a user or organisational goal.
6. **Human developers to machine consumers**
APIs are increasingly discovered and invoked by automation and AI systems, raising the importance of machine-readable constraints, authority, provenance and auditability.

Where UK government is now

There is a clear difference between **policy direction** and **current maturity**.

The 2025 Blueprint for Modern Digital Government proposes a Digital Backbone, a standard set of APIs and events across public-sector organisations, and an expectation that every new central government service will have an open API. It also proposes a “once only” rule under which information already supplied to one service can be reused by another with appropriate safeguards. ([GOV.UK](#))

However, cross-government research published in April 2025 found that API readiness and maturity varied significantly and was generally low. Problems included limited documentation, inconsistent application of standards, legacy and manual processes, security and governance concerns, and fragmented discoverability. ([Data in Government](#))

The subsequent API Hub alpha identified demand for:

- a refreshed API catalogue;
- API quality and security scores;
- clearer role-based guidance;
- practical developer tooling;
- a signed-in developer experience. ([Data in Government](#))

A vital distinction is:

Open API should mean discoverable, documented and available under transparent conditions—not necessarily anonymous access or open data.

The existing catalogue explicitly notes that inclusion does not mean that an API is publicly accessible; APIs may retain their own licensing, authorisation and access restrictions. ([API Catalogue](#))

Future direction possibilities

These are possibilities and strategic choices, rather than predictions.

1. From an API catalogue to an active control plane

High-confidence direction.

A future government API platform could go beyond listing APIs and continuously expose:

- accountable owner and support route;
- OpenAPI or AsyncAPI description;
- data classification and access conditions;
- conformance and security results;
- service-level objectives and current health;
- versions, deprecation dates and change notices;
- known consumers and dependencies;
- lineage and provenance.

The catalogue would become part of delivery and governance rather than a manually maintained website.

Action now: require discoverable ownership and machine-readable contracts as conditions of production deployment.

2. Treat API, event and workflow descriptions as one family

High-confidence direction.

A mature public capability might have:

- **OpenAPI** for request–response operations;
- **AsyncAPI** for commands, events and subscriptions;
- **Arazzo** for the sequences needed to achieve an outcome;

- shared schemas, identifiers and policy metadata across all three.

This is particularly relevant to life events. “Tell government someone has died” is not one endpoint; it is a governed sequence of notifications, checks, decisions and updates across organisational boundaries.

Action now: design around business events and user outcomes before selecting the interaction protocol.

3. Agent-ready, but not agent-trusting

Emerging and less certain.

AI agents may increasingly discover and invoke government capabilities. A safe government approach would require more than wrapping existing endpoints in MCP or A2A. It would need:

- identities for non-human actors;
- tightly scoped and time-limited authority;
- explicit distinction between reading, recommending and acting;
- human approval at proportionate decision points;
- complete audit trails;
- idempotency and safe retry behaviour;
- transaction limits and policy checks;
- compensating actions or rollback;
- reliable citations and provenance;
- clear routes for human challenge and redress.

The key principle is:

An agent’s ability to understand an API must never be mistaken for authority to use it.

Agent authority and the legal accountability layer — law, not protocol

Not legal advice — a discussion summary. Confirm specifics with departmental legal advisers and current ICO guidance; several Data (Use and Access) Act 2025 provisions commence by regulation, so check what is in force at deployment.

The controls above answer two technical questions well — **can the agent act, and can we prove what it did?**

They do not answer the questions a public body is legally accountable for: **may government lawfully take this decision, in this way, and can the person affected understand and challenge it?** Those duties are set by law and by public-law principles that bind regardless of the protocol and largely cannot be delegated to a system. The agentic profile should therefore be paired with a legal-control layer; in several places the binding control is law, not protocol.

What the law requires, and where it lands in the agent stack

Data protection and automated decisions — UK GDPR / DPA 2018, as amended by the Data (Use and Access) Act 2025 (new Arts 22A–22D, in force 5 February 2026). For a significant, solely-automated decision (legal or similarly significant effect, with no meaningful human involvement): give the person information, allow representations, enable human intervention or review, and let them contest it. Special-category data stays tightly restricted. [maps to] approval design; human-in-the-loop; a documented meaningful-human-involvement test per use case.

DPIA and data protection by design — UK GDPR Arts 35 and 25: high-risk processing needs a Data Protection Impact Assessment before go-live, with privacy and data minimisation designed in, not bolted on. [maps to] a mandatory pre-production gate.

Algorithmic transparency (ATRS) — the Algorithmic Transparency Recording Standard is mandatory across central government and tied to digital spend controls: publish a record for algorithmic tools that significantly influence decisions with a public effect or that interact with the public. [maps to] registry/inventory; a procurement and spend-control gate.

Public-law duties — lawfulness and vires, rationality, procedural fairness, and the duty to give reasons. **A named human remains the legal decision-maker even when an agent executes the steps, and decisions must**

be explainable enough to withstand challenge. [maps to] identity that records human authority; evidence that captures the reasons relied on.

Equality — the Public Sector Equality Duty (Equality Act 2010, s.149): due regard to eliminating discrimination and advancing equality, with an equality impact assessment and bias testing across protected groups. [maps to] evaluation; pre-production gate.

Redress and accountability — **a clear route for the citizen to obtain an explanation, human review and appeal; complaint handling; exposure to judicial review; and unambiguous ownership.** [maps to] incident response; governance and portfolio management.

Every row maps onto a control the above agent-readiness list which already proposes (identity, scoped authority, approval, evidence, registry, evaluation, redress). The point raised here is to make the legal duty the reason for each control, and to add the few currently missing: the significant-decision test, the DPIA and equality-impact gate, the named accountable decision-maker, and the citizen redress route.

Decision questions to add

1. **Significant-decision test:** for each use case, is the action a significant decision under DUAA Arts 22A–22D, and is there meaningful human involvement — or are the Art 22C safeguards (information, representations, intervention, contest) in place?
2. **Transparency:** which agentic tools require a published ATRS record, and is that a pre-production gate tied to spend controls?
3. **Accountable decision-maker:** who is the named human legally responsible for each agent decision, and how is that authority recorded on every action?
4. **Redress:** what is the citizen route to an explanation, human review and appeal when an agent action affects them, and where is it logged?
5. **Impact assessments:** are a DPIA and an equality impact assessment mandatory gates before any rights-, eligibility- or money-affecting agent goes live?

Action now: pair the agentic profile with this legal-control layer, and mark on every tool where the binding control is law, not protocol.

4. Semantic interoperability becomes more important than syntactic consistency

High strategic importance, difficult execution.

Two departments can both publish perfect JSON and still disagree about what a household, address, entitlement, organisation, decision or case means.

The next stage is likely to emphasise:

- persistent identifiers;
- shared vocabularies and code lists;
- explicit definitions and provenance;
- mappings between domain models;
- effective dates and temporal meaning;
- authoritative-source declarations.

A single universal government data model would probably become unmanageable. A federated model—common foundations with domain-owned semantics—is more plausible.

5. Move from bearer access towards demonstrable authority

High-confidence security direction.

Future API security will need to distinguish:

- the person or organisation on whose behalf an action occurs;
- the software or workload making the request;
- the delegation chain;
- the particular purpose and permitted action;
- the evidence supporting the decision.

This suggests greater use of workload identity, proof-bound credentials, fine-grained policy, short-lived access and continuous audit rather than long-lived shared secrets.

For government, authorisation is not merely “is this token valid?” It is:

“Who is acting, for whom, for what purpose, using which legal authority, and what evidence must be retained?”

6. APIs become enduring public products

Primarily an operating-model decision.

The hardest API problems are often institutional rather than technical:

- no funded owner;
- undocumented dependencies;
- breaking changes without notice;
- suppliers retaining essential knowledge;
- service teams optimising only for their own front end;
- APIs treated as project outputs rather than continuing public infrastructure.

A government API should therefore have the characteristics of a maintained public product: ownership, user research, support, reliability targets, lifecycle policy, backward-compatibility commitments and sustainable funding.

References

Sources verified against primary or authoritative pages; accessed 20 June 2026. URLs are shown without tracking parameters. Numbering follows the order of citation in the timeline.

- [1] CERN. A short history of the Web (Web released royalty-free, 30 April 1993).
<https://home.cern/science/computing/birth-web/short-history-web>
- [2] W3C. The Original HTTP as defined in 1991 (HTTP/0.9).
<https://www.w3.org/Protocols/HTTP/AsImplemented.html>
- [3] IETF / RFC Editor. RFC 1945, Hypertext Transfer Protocol HTTP/1.0, May 1996.
<https://www.rfc-editor.org/info/rfc1945>
- [4] IETF / RFC Editor. RFC 2068, Hypertext Transfer Protocol HTTP/1.1, January 1997. <https://www.rfc-editor.org/info/rfc2068>
- [5] IETF / RFC Editor. RFC 2616, Hypertext Transfer Protocol HTTP/1.1 (revision), June 1999. <https://www.rfc-editor.org/info/rfc2616>
- [6] IETF / RFC Editor. RFC 7230–7235, HTTP/1.1 refactor, June 2014. <https://www.rfc-editor.org/info/rfc7230>
- [7] IETF / RFC Editor. RFC 7540, HTTP/2, May 2015. <https://www.rfc-editor.org/info/rfc7540>
- [8] IETF / RFC Editor. RFC 9110, HTTP Semantics, June 2022. <https://www.rfc-editor.org/info/rfc9110>
- [9] IETF / RFC Editor. RFC 9112 (HTTP/1.1), RFC 9113 (HTTP/2), RFC 9114 (HTTP/3), June 2022.
<https://www.rfc-editor.org/info/rfc9114>
- [10] W3C. Extensible Markup Language (XML) 1.0, Recommendation, 10 February 1998.
<https://www.w3.org/TR/1998/REC-xml-19980210>
- [11] W3C. SOAP 1.1, Note, 8 May 2000. <https://www.w3.org/TR/2000/NOTE-SOAP-20000508>
- [12] W3C. Web Services Description Language (WSDL) 1.1, Note, 15 March 2001.
<https://www.w3.org/TR/2001/NOTE-wsdl-20010315>
- [13] R. T. Fielding. Architectural Styles and the Design of Network-based Software Architectures (REST), PhD dissertation, University of California, Irvine, 2000.
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [14] IETF / RFC Editor. RFC 4627, application/json Media Type, July 2006 — obsoleted by RFC 8259 (STD 90), December 2017. <https://www.rfc-editor.org/info/rfc8259>
- [15] eBay Inc. Celebrating 20 years of eBay APIs (first APIs launched 2000).
<https://innovation.ebayinc.com/stories/celebrating-20-years-ebays-new-apis-enable-developers-to-create-modern-buying-and-selling-experiences/>
- [16] IETF / RFC Editor. RFC 6749, The OAuth 2.0 Authorization Framework, October 2012. <https://www.rfc-editor.org/info/rfc6749>
- [17] Government Digital Service. Putting APIs first: legislation.gov.uk, 30 March 2012.
<https://gds.blog.gov.uk/2012/03/30/putting-apis-first-legislation-gov-uk/>
- [18] OpenAPI Initiative. About the OpenAPI Initiative (formed November 2015; Swagger 2.0 donation; OpenAPI 3.0 in 2017). <https://www.openapis.org/about>
- [19] GraphQL Foundation. GraphQL (used at Facebook from 2012; open-sourced 2015). <https://graphql.org/>
- [20] gRPC (CNCF). High-performance RPC over Protocol Buffers. <https://grpc.io/>
- [21] AsyncAPI Initiative. Specification for event-driven APIs (created 2017). <https://www.asyncapi.com/>
- [22] GDS. API technical and data standards (first published February 2018). <https://www.gov.uk/guidance/gds-api-technical-and-data-standards>
- [23] GDS. Open Standards Board recommendations: CSVs and API descriptions (OpenAPI 3), 2 October 2019.
<https://technology.blog.gov.uk/2019/10/02/improve-csvs-and-api-descriptions-with-these-open-standards-board-recommendations/>
- [24] CNCF. Serverless specification CloudEvents reaches version 1.0, 28 October 2019.
<https://www.cncf.io/announcements/2019/10/28/serverless-specification-cloudevents-reaches-version-1-0/>
- [25] AsyncAPI Initiative. AsyncAPI 3.0 (November 2023) and 3.1.0 (31 January 2026).
<https://www.asyncapi.com/blog/release-notes-3.1.0>
- [26] OpenAPI Initiative. Announcing OpenAPI v3.2, 23 September 2025; spec v3.2.0.
<https://spec.openapis.org/oas/v3.2.0.html>
- [27] OpenAPI Initiative. Arazzo Specification v1.1.0 (schema iteration dated 2026-04-15; adds AsyncAPI support). <https://spec.openapis.org/arazzo/v1.1.0.html>

- [28] IETF / RFC Editor. RFC 9700, Best Current Practice for OAuth 2.0 Security, January 2025. <https://www.rfc-editor.org/info/rfc9700>
- [29] Model Context Protocol (Anthropic). Introduced November 2024; specification 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>
- [30] Linux Foundation. Linux Foundation Launches the Agent2Agent (A2A) Protocol Project, 23 June 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
- [31] GOV.UK / DSIT. A blueprint for modern digital government, January 2025. <https://www.gov.uk/government/publications/a-blueprint-for-modern-digital-government>
- [32] Data in Government. Joining up the dots: findings of our recent API discovery, 3 April 2025. <https://dataingovernment.blog.gov.uk/2025/04/03/joining-up-the-dots-the-findings-of-our-recent-api-discovery/>
- [33] Data in Government. Strengthening and extending connectivity: what we learned from the API Hub alpha, 28 November 2025. <https://dataingovernment.blog.gov.uk/2025/11/28/strengthening-and-extending-connectivity-what-we-learned-from-the-api-hub-alpha/>
- [34] GOV.UK API Catalogue. Inclusion does not imply public accessibility; APIs retain their own licensing and access conditions. <https://www.api.gov.uk/>